DATABASE MANAGEMENT LAB

LAB Report

Submitted in partial fulfilment of the requirement of the degree of

BACHELORS OF TECHNOLOGY

to

K.R Mangalam University

Submitted by:

MAHIN

(2401010123)

B.Tech CSE (Core) SecA

Semester 4th

Submitted to

Ms. Mansi Kajal

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001, India

Batch – 2024-2028

Lab Experiment 1

AIM: Design an Entity-Relationship (ER) diagram and convert it into relational tables for a College Database scenario.

COLLEGE DATABASE

STUDENT (USN, SName, Address, Phone, Gender)
SEMSEC (SSID, Sem, Sec)
CLASS (USN, SSID)
SUBJECT (Subcode, Title, Sem, Credits)
IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

SOLUTION

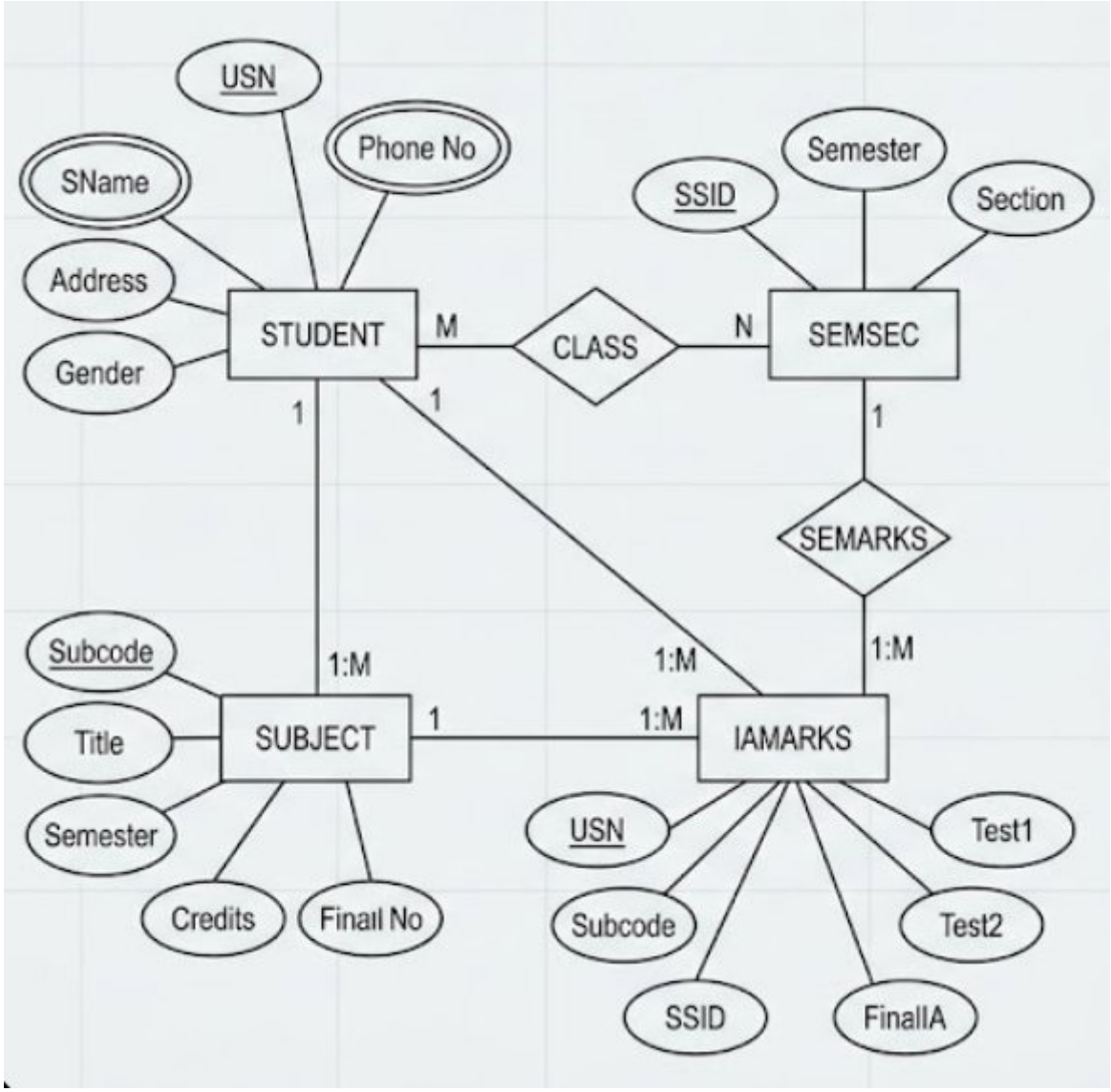
I. **Identify Entities**: The core entities are **STUDENT**, **SEMSEC**, **SUBJECT**, and **IAMARKS**, represented using rectangles.

II. **Define Attributes:** Attributes are represented using ellipses. Primary keys are identified by underlining the attribute name.
STUDENT: USN (Primary Key), SName, Address, Phone No, Gender.
SEMSEC: SSID (Primary Key), Semester, Section.
SUBJECT: Subcode (Primary Key), Title, Semester, Credits.
IAMARKS: USN, Subcode, SSID, Test1, Test2, Test3, FinalIA.

III. **Multivalued Attributes**: **Phone No** is shown using a double ellipse, indicating a student may have multiple contact numbers.

IV. **Relationships:**
a) **CLASS**: A diamond-shaped relationship between STUDENT and SEMSEC with an **M:N** cardinality.
b) **IAMARKS (Associative Entity)**: Resolves the **M:N** relationship between STUDENT and SUBJECT.
c) **Cardinality**: Relationships between STUDENT-IAMARKS, SUBJECT-IAMARKS, and SEMSEC-IAMARKS are all **1:M**

V. **ER Diagram:**


USN
Phone No
SName
Address
Gender
STUDENT
M
CLASS
N
SEMSEC
SSID
Semester
Section
1
1
1
SEMARKS
1:M
1:M
1:M
Subcode
Title
Semester
SUBJECT
1
1:M
IAMARKS
Credits
Finail No
USN
Subcode
SSID
FinalIA
Test1
Test2

Lab Experiment 2

AIM: Design an Entity-Relationship (ER) diagram and convert it into relational tables for a Company Database scenario.

COMPANY DATABASE

EMPLOYEE (SSN, Name, Address, Salary, Sex, SuperSSN, DNO)
DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate)
PROJECT (PNo, PName, PLocation, DNO)
DLOCATION (DNo, DLoc)
WORKS_ON (SSN, PNo, Hours)
MANAGES (SSN, DNo)

SOLUTION

I. **Identify Entities**: The core entities include **EMPLOYEE**, **DEPARTMENT**, **PROJECT**, and **DLOCATION**.

II. **Define Attributes:** Attributes are represented using ellipses. Primary keys are identified by underlining the attribute name.

EMPLOYEE: SSN (Primary Key), Name, Address, Sex, Salary, SuperSSN, DNo.
DEPARTMENT: DNo (Primary Key), DName, MgrSSN, MgrStartDate.
PROJECT: PNo (Primary Key), PName, PLocation, DNo.
DLOCATION: DNo, DLoc.

III. **Primary Keys:** All primary keys are underlined to ensure data integrity.

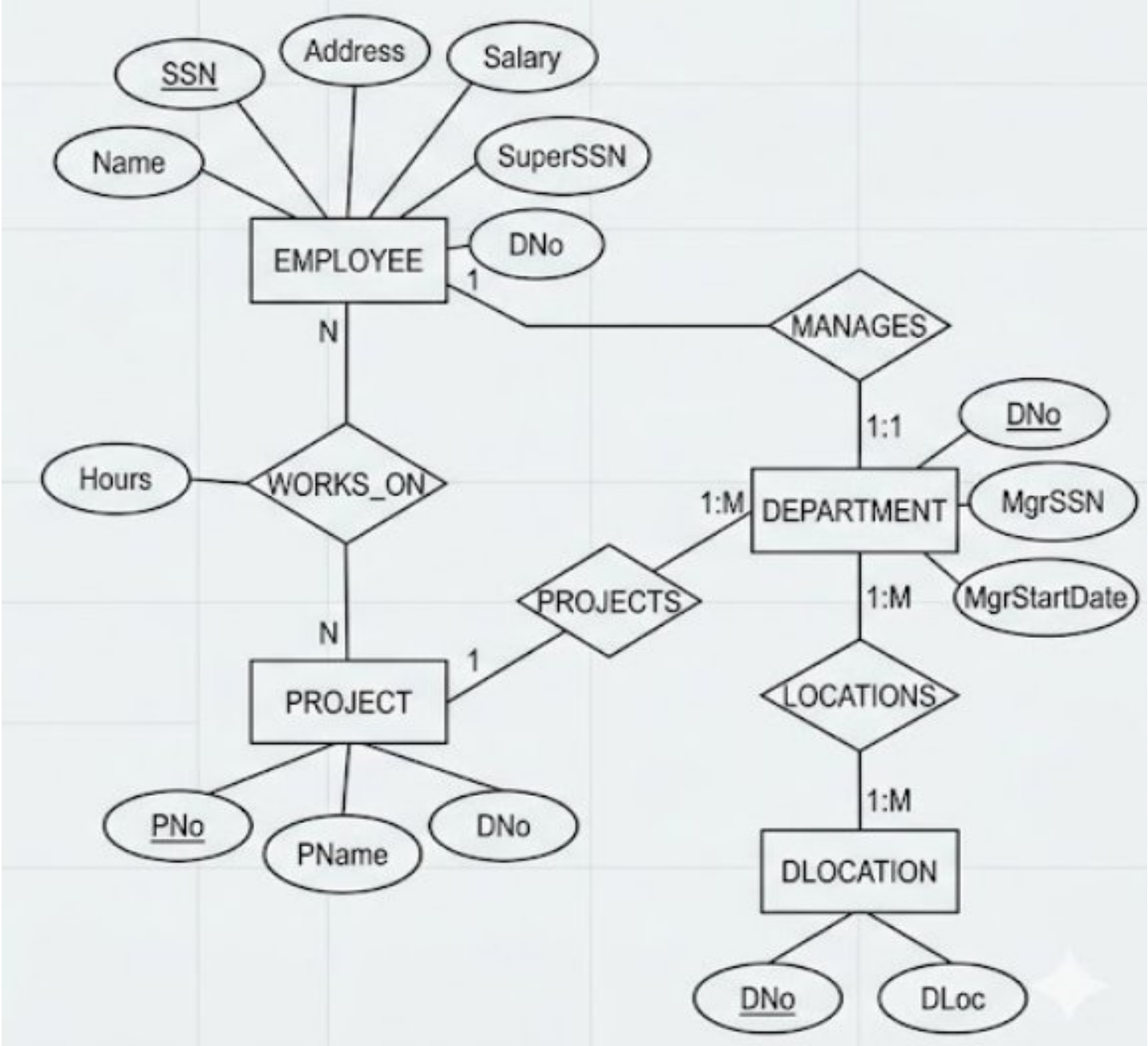
IV. **Relationships:**

a) **WORKS_ON:** An M:N relationship between EMPLOYEE and PROJECT, including the attribute Hours.

b) **MANAGES:** A 1:1 relationship between EMPLOYEE and DEPARTMENT.

c) **Departmental Connections: A 1:M relationship exists from DEPARTMENT to both PROJECT and DLOCATION.**

V. **ER Diagram:**


SSN
Address
Salary
Name
SuperSSN
EMPLOYEE
DNo
1
MANAGES
N
1:1
DNo
Hours
WORKS_ON
1:M
DEPARTMENT
MgrSSN
PROJECTS
1:M
MgrStartDate
N
1
PROJECT
LOCATIONS
1:M
PNo
PName
DNo
DLOCATION
DNo
DLoc

Lab Experiment 3

AIM: Database Schema for a customer-sale scenario:

Customer(Cust id : integer, cust_name: string)
Item(item_id: integer, item_name: string, price: integer)
Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following:

SOLUTION CODE: (By MAHIN-2401010123)

(a) Create tables with the appropriate integrity constant.

```
CREATE DATABASE IF NOT EXISTS CustomerSaleDB;
USE CustomerSaleDB;
DROP TABLE IF EXISTS Sale;
DROP TABLE IF EXISTS Item;
DROP TABLE IF EXISTS Customers;
CREATE TABLE Customers (
   cust_id INT PRIMARY KEY,
   cust_name VARCHAR(50) NOT NULL
);
CREATE TABLE Item (
   item_id INT PRIMARY KEY,
   item_name VARCHAR(50) NOT NULL,
   price INT CHECK (price > 0)
);
CREATE TABLE Sale (
   bill_no INT PRIMARY KEY,
   bill_date DATE NOT NULL,
   cust_id INT,
   item_id INT,
   qty_sold INT CHECK (qty_sold > 0),
   FOREIGN KEY (cust_id) REFERENCES Customers(cust_id),
   FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

(b) Insert 10 records in each table.

```
INSERT INTO Customers (cust_id, cust_name) VALUES
(1, 'ayaan'), (2, 'sana'), (3, 'vihaan'), (4, 'zoya'), (5, 'advik'),
(6, 'kiara'), (7, 'arjun'), (8, 'ritika'), (9, 'ishaan'), (10, 'ananya');
```

cust_id	cust_name
1	ayaan
2	sana
3	vihaan
4	zoya
5	advik
6	kiara
7	arjun
8	ritika
9	ishaan
10	ananya
NULL	NULL

```
INSERT INTO Item (item_id, item_name, price) VALUES
(101, 'backpack', 1200), (102, 'bottle', 450), (103, 'diary', 150),
(104, 'lamp', 850), (105, 'mouse', 600), (106, 'keyboard', 1100),
(107, 'headphones', 2500), (108, 'watch', 3500), (109, 'charger', 400),
(110, 'stand', 300);
```

item_id	item_name	price
101	backpack	1200
102	bottle	450
103	diary	150
104	lamp	850
105	mouse	600
106	keyboard	1100
107	headphones	2500
108	watch	3500
109	charger	400
110	stand	300
NULL	NULL	NULL

```
INSERT INTO Sale (bill_no, bill_date, cust_id, item_id, qty_sold) VALUES
(1, CURDATE(), 1, 103, 1),
(2, CURDATE(), 2, 101, 3),
(3, CURDATE(), 5, 105, 2),
(4, '2026-04-01', 3, 104, 1),
(5, '2026-04-02', 4, 106, 1),
(6, '2026-04-03', 5, 102, 4),
(7, CURDATE(), 6, 110, 1),
(8, '2026-04-04', 7, 108, 2),
(9, CURDATE(), 5, 109, 5),
(10, '2026-04-04', 8, 107, 1);
```

bill_no	bill_date	cust_id	item_id	qty_sold
1	2026-04-05	1	103	1
2	2026-04-05	2	101	3
3	2026-04-05	5	105	2
4	2026-04-01	3	104	1
5	2026-04-02	4	106	1
6	2026-04-03	5	102	4
7	2026-04-05	6	110	1
8	2026-04-04	7	108	2
9	2026-04-05	5	109	5
10	2026-04-04	8	107	1
NULL	NULL	NULL	NULL	NULL

(c) List bills for current date with customer names and item numbers.

```
SELECT S.bill_no, C.cust_name, S.item_id
FROM Sale S
JOIN Customers C ON S.cust_id = C.cust_id
WHERE S.bill_date = CURDATE();
```

bill_no	cust_name	item_id
1	ayaan	103
2	sana	101
3	advik	105
7	kiara	110
9	advik	109

(d) List total Bill details with qty, price, and final amount.

```
SELECT S.bill_no, C.cust_name, I.item_name,
S.qty_sold, I.price, (S.qty_sold * I.price)
AS Final_Amount
FROM Sale S
JOIN Customers C ON S.cust_id = C.cust_id
JOIN Item I ON S.item_id = I.item_id;
```

bill_no	cust_name	item_name	qty_sold	price	Final_Amount
1	ayaan	diary	1	150	150
2	sana	backpack	3	1200	3600
4	vihaan	lamp	1	850	850
5	zoya	keyboard	1	1100	1100
3	advik	mouse	2	600	1200
6	advik	bottle	4	450	1800
9	advik	charger	5	400	2000
7	kiara	stand	1	300	300
8	arjun	watch	2	3500	7000
10	ritika	headphones	1	2500	2500

(e) List details of customers who bought products with price > 200.

```
SELECT DISTINCT C.* FROM Customers C
JOIN Sale S ON C.cust_id = S.cust_id
JOIN Item I ON S.item_id = I.item_id
WHERE I.price > 200;
```

cust_id	cust_name
2	sana
5	advik
3	vihaan
4	zoya
8	ritika
7	arjun
6	kiara

(f) Count of products bought by each customer.

```
SELECT C.cust_name, SUM(S.qty_sold)
AS Total_Products_Bought
FROM Customers C
JOIN Sale S ON C.cust_id = S.cust_id
GROUP BY C.cust_name;
```

cust_name	Total_Products_Bought
ayaan	1
sana	3
vihaan	1
zoya	1
advik	11
kiara	1
arjun	2
ritika	1

(g) Products bought by customer with cust_id 5.

```
SELECT I.item_name, S.qty_sold
FROM Sale S
JOIN Item I ON S.item_id = I.item_id
WHERE S.cust_id = 5;
```

item_name	qty_sold
mouse	2
bottle	4
charger	5

(h) List item details sold as of today.

```
SELECT I.* FROM Sale S
JOIN Item I ON S.item_id = I.item_id
WHERE S.bill_date = CURDATE();
```

item_id	item_name	price
103	diary	150
101	backpack	1200
105	mouse	600
110	stand	300
109	charger	400

(i) Create view for bill details.

```
CREATE OR REPLACE VIEW Bill_Details AS
SELECT S.bill_no, S.bill_date, S.cust_id, S.item_id, I.price, S.qty_sold,
    (S.qty_sold * I.price) AS Amount
FROM Sale S
JOIN Item I ON S.item_id = I.item_id;
```

bill_no	bill_date	cust_id	item_id	price	qty_sold	Amount
1	2026-04-05	1	103	150	1	150
2	2026-04-05	2	101	1200	3	3600
3	2026-04-05	5	105	600	2	1200
4	2026-04-01	3	104	850	1	850
5	2026-04-02	4	106	1100	1	1100
6	2026-04-03	5	102	450	4	1800
7	2026-04-05	6	110	300	1	300
8	2026-04-04	7	108	3500	2	7000
9	2026-04-05	5	109	400	5	2000
10	2026-04-04	8	107	2500	1	2500

(j) Create view for weekly daily sales.

```
SELECT bill_date, SUM(S.qty_sold * I.price) AS Total_Daily_Sales
FROM Sale S
JOIN Item I ON S.item_id = I.item_id
WHERE bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY bill_date;
```

bill_date	Total_Daily_Sales
2026-04-05	7250
2026-04-01	850
2026-04-02	1100
2026-04-03	1800
2026-04-04	9500

Lab Experiment 4.1

AIM: Consider the following table: Table: CLASS (ID,Name)

```sql
CREATE DATABASE IF NOT EXISTS school_db;
USE school_db;
SET SQL_SAFE_UPDATES = 0;

DROP TABLE IF EXISTS student_class;
CREATE TABLE student_class (
    roll_id INT,
    student_name VARCHAR(20)
);

INSERT INTO student_class VALUES (1, 'rohit'), (2, 'sara'), (4, 'nikhil');
```

roll_id	student_name
1	rohit
2	sara
4	nikhil

SOLUTION CODE: (By MAHIN-2401010123)

(a) INSERT INTO class VALUES (5,mahin');.

```sql
SET autocommit = 0;
START TRANSACTION;
INSERT INTO student_class VALUES (5, 'mahin');
COMMIT;
```

roll_id	student_name
1	rohit
2	sara
4	nikhil
5	mahin

(b) Update student_name with Savepoints A,B, and C

```sql
START TRANSACTION;
UPDATE student_class SET student_name = 'abhishek' WHERE roll_id = 5;
SAVEPOINT pt_a;

INSERT INTO student_class VALUES (6, 'tanya');
SAVEPOINT pt_b;

INSERT INTO student_class VALUES (7, 'rohit');
SAVEPOINT pt_c;
```

- **Select 1: Shows all 5 records**

```sql
SELECT * FROM student_class;
ROLLBACK TO pt_b;
```

roll_id	student_name
1	rohit
2	sara
4	nikhil
5	abhishek
6	tanya
7	rohit

- **Select 2: id 7 is gone; id 6 remains**

```sql
SELECT * FROM student_class;
ROLLBACK TO pt_a;
```

roll_id	student_name
1	rohit
2	sara
4	nikhil
5	abhishek
6	tanya

- **Select 3: id 6 is gone; id 5 remains as 'abhishek'**

```sql
SELECT * FROM student_class;
COMMIT;
```

roll_id	student_name
1	rohit
2	sara
4	nikhil
5	abhishek

Lab Experiment 4.2

AIM: Consider the following table and perform the following TCL-based SQL script:

acc_id	holder_name	curr_balance
101	ishan	5000
102	megha	7000
103	daksh	6000

a) Insert data into the table: (104, 'kavya', 4000)

```
START TRANSACTION;
INSERT INTO bank_ledger VALUES (104, 'kavya', 4000);
SET autocommit = 0;
SAVEPOINT s1;
```

acc_id	holder_name	curr_balance
101	ishan	5000
102	megha	7000
103	daksh	6000
104	kavya	4000

b) Update table, set balance where acc_id = 101 and acc_id = 102

```
UPDATE bank_ledger
SET curr_balance = curr_balance + 1500 WHERE acc_id = 101;
UPDATE bank_ledger
SET curr_balance = curr_balance - 1000 WHERE acc_id = 102;
SAVEPOINT s2;
```

acc_id	holder_name	curr_balance
101	ishan	6500
102	megha	6000
103	daksh	6000
104	kavya	4000

c) Delete from table, where acc_id = 103 and insert 'rehan' at acc_id = 105

```
DELETE FROM bank_ledger WHERE acc_id = 103;
INSERT INTO bank_ledger VALUES (105, 'rehan', 3000);
SAVEPOINT s3;
```

acc_id	holder_name	curr_balance
101	ishan	6500
102	megha	6000
104	kavya	4000
105	rehan	3000

d) Undoes the delete of daksh, the insert of rehan, and name change of megha.

```
ROLLBACK TO s2;
SELECT * FROM bank_ledger;
COMMIT;
```

acc_id	holder_name	curr_balance
101	ishan	6500
102	megha	6000
103	daksh	6000
104	kavya	4000

e) Insert VALUES(106, 'zoya', 4500) and SAVEPOINT s4.

```
INSERT INTO bank_ledger VALUES (106, 'zoya', 4500);
SAVEPOINT s4;
```

acc_id	holder_name	curr_balance
101	ishan	6500
102	megha	6000
103	daksh	6000
104	kavya	4000
106	zoya	4500

f) Update table, set balance where acc_id = 101 and 104.

```
UPDATE bank_ledger SET curr_balance = curr_balance + 500
WHERE acc_id IN (101, 104);
SELECT * FROM bank_ledger;
```

acc_id	holder_name	curr_balance
101	ishan	7000
102	megha	6000
103	daksh	6000
104	kavya	4500
106	zoya	4500

g) ROLLBACK to s4 and commit.

```
ROLLBACK TO s4;
SELECT * FROM bank_ledger;
COMMIT;
```

acc_id	holder_name	curr_balance
101	ishan	6500
102	megha	6000
103	daksh	6000
104	kavya	4000
106	zoya	4500

Lab Experiment 5

AIM: Create EMPLOYEES (Employee_Id, First_Name, Last_Name, Email, Phone_Number, Hire_Date, Job_Id, Salary, Commission_Act, Manager_Id, Department_Id)

(a) Find out the employee id, names, salaries of all the employees

(b) List out the employees who works under manager 100

(c) Find the names of the employees who have a salary greater than or equal to 4800

(d) List out the employees whose last name is 'austin'

(e) Find the names of the employees who works in departments 60,70 and 80

(f) Display the unique ManagerId.

SOLUTION CODE: (By MAHIN-2401010123)

```
CREATE DATABASE IF NOT EXISTS StaffRegistry;
USE StaffRegistry;

DROP TABLE IF EXISTS Employees;

CREATE TABLE Employees (
   EmployeeId INT,
   FirstName VARCHAR(50),
   LastName VARCHAR(50),
   EmailAddr VARCHAR(50),
   PhoneNum VARCHAR(20),
   HireDate DATE,
   JobId VARCHAR(20),
   SalaryAmt INT,
   CommPct INT,
   ManagerId INT,
   DeptId INT
);

INSERT INTO Employees (EmployeeId, FirstName, LastName, EmailAddr, PhoneNum, HireDate, JobId,
SalaryAmt, CommPct, ManagerId, DeptId) VALUES
(201, 'arnav', 'sharma', 'arnav@mail.com', '99881', '2020-05-20', 'IT_PROG', 5500, NULL, 100, 60),
(202, 'diya', 'iyer', 'diya@mail.com', '99882', '2021-06-15', 'HR_REP', 6200, NULL, 100, 70),
(203, 'kunal', 'austin', 'kunal@mail.com', '99883', '2019-02-10', 'FI_ACC', 4200, NULL, 101, 80),
(204, 'aditi', 'patel', 'aditi@mail.com', '99884', '2022-01-30', 'IT_PROG', 7500, NULL, 100, 60),
(205, 'rahul', 'austin', 'rahul@mail.com', '99885', '2023-03-12', 'HR_REP', 4800, NULL, 102, 90);
```

(a) Find out the employee id, names, salaries of all the employees.

```
SELECT EmployeeId, FirstName, LastName, SalaryAmt
FROM Employees;
```

(b) List out the employees who works under manager 100

```
SELECT * FROM Employees;
WHERE ManagerId = 100;
```

(c) Find the names of the employees who have a salary greater than or equal to 4800.

```
SELECT FirstName, LastName
FROM Employees
WHERE SalaryAmt >= 4800;
```

(d) List out the employees whose last name is 'AUSTIN'.

```
SELECT * FROM Employees
WHERE LastName = 'austin';
```

(e) Find the names of the employees who works in departments 60, 70 and 80.

```
SELECT FirstName, LastName
FROM Employees
WHERE DeptId IN (60, 70, 80);
```

(f) Display the unique Manager_Id.

```
SELECT DISTINCT ManagerId
FROM Employees;
```

OUTPUT

EmployeeId	FirstName	LastName	SalaryAmt
201	arnav	sharma	5500
202	diya	iyer	6200
203	kunal	austin	4200
204	aditi	patel	7500
205	rahul	austin	4800

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeId	FirstName	LastName	EmailAddr	PhoneNum	HireDate	JobId	SalaryAmt	CommPct	ManagerId	DeptId
201	arnav	sharma	arnav@mail.com	99881	2020-05-20	IT_PROG	5500	NULL	100	60
202	diya	iyer	diya@mail.com	99882	2021-06-15	HR_REP	6200	NULL	100	70
204	aditi	patel	aditi@mail.com	99884	2022-01-30	IT_PROG	7500	NULL	100	60

FirstName	LastName
arnav	sharma
diya	iyer
aditi	patel
rahul	austin

EmployeeId	FirstName	LastName	EmailAddr	PhoneNum	HireDate	JobId	SalaryAmt	CommPct	ManagerId	DeptId
203	kunal	austin	kunal@mail.com	99883	2019-02-10	FI_ACC	4200	NULL	101	80
205	rahul	austin	rahul@mail.com	99885	2023-03-12	HR_REP	4800	NULL	102	90

FirstName	LastName
arnav	sharma
diya	iyer
kunal	austin
aditi	patel

ManagerId
100
101
102

Lab Experiment 6

AIM: Create Client_master with the following fields (ClientNO, Name, Address, City, State, bal_due)

a) Insert five records.
b) Find the names of clients whose bal_due> 5000 .
c) Change the bal_due of ClientNO " C123" to Rs. 5100.
d) Change the name of Client_master to Client12.
e) Display the bal_due heading as "BALANCE".

SOLUTION CODE: (By MAHIN-2401010123)

```
CREATE DATABASE IF NOT EXISTS LabDatabase;
USE LabDatabase;

DROP TABLE IF EXISTS ClientMaster;
DROP TABLE IF EXISTS Client12;

CREATE TABLE ClientMaster (
   ClientNo VARCHAR(10),
   FullName VARCHAR(50),
   UserAddress VARCHAR(100),
   CityName VARCHAR(50),
   StateName VARCHAR(50),
   BalanceDue INT
);
```

(a) Insert five records.

```
INSERT INTO ClientMaster (ClientNo, FullName, UserAddress, CityName, StateName, BalanceDue) VALUES
('c901', 'ishan malhotra', '142 palika bazaar', 'new delhi', 'delhi', 4800),
('c902', 'meera nair', 'pinnacle tower 402', 'thiruvananthapuram', 'kerala', 7200),
('c123', 'arjun kapoor', 'model town lane 4', 'ludhiana', 'punjab', 3000),
('c904', 'sana mirza', 'hitech city road', 'hyderabad', 'telangana', 9500),
('c905', 'aditya joshi', 'near lakeside park', 'bhopal', 'madhya pradesh', 1200);
```

(b) Clients with balance due > 5000

```
SELECT FullName
FROM ClientMaster
WHERE BalanceDue > 5000;
```

(c) Change/Update balance for client 'c123' to 5100

```
UPDATE ClientMaster
SET BalanceDue = 5100
WHERE ClientNo = 'c123';
SELECT * FROM ClientMaster;
```

(d) Change the name of the table to Client12.

```
RENAME TABLE ClientMaster TO Client12;
```

(e) Display the bal_due heading as “BALANCE”.

```
SELECT BalanceDue AS BALANCE
FROM Client12;
```

OUTPUT

FullName
meera nair
sana mirza

ClientNo	FullName	UserAddress	CityName	StateName	BalanceDue
c901	ishan malhotra	142 palika bazaar	new delhi	delhi	4800
c902	meera nair	pinnacle tower 402	thiruvananthapuram	kerala	7200
c123	arjun kapoor	model town lane 4	ludhiana	punjab	5100
c904	sana mirza	hitech city road	hyderabad	telangana	9500
c905	aditya joshi	near lakeside park	bhopal	madhya pradesh	1200

BALANCE
4800
7200
5100
9500
1200

Lab Experiment 7

Objective: Rollback and Commit commands Create Teacher table with the following fields (Name, DeptNo, Date of joining, DeptName, Location, Salary)

a) Insert five records.
b) Give Increment of 25% salary for Mathematics Department.
c) Perform Rollback command.
d) Give increment of 15% salary for Commerce Department.
e) Perform Commit command.

SOLUTION CODE: (By MAHIN-2401010123)

```
CREATE DATABASE IF NOT EXISTS school_management;
USE school_management;
SET SQL_SAFE_UPDATES = 0;

DROP TABLE IF EXISTS Teacher;
CREATE TABLE Teacher (
    TeacherName VARCHAR(50),
    DeptNo INT,
    DateOfJoining DATE,
    DeptName VARCHAR(50),
    LocationName VARCHAR(50),
    SalaryAmt INT
);
```

(a) Insert five records.

```
INSERT INTO Teacher (TeacherName, DeptNo, DateOfJoining, DeptName, LocationName, SalaryAmt)
  VALUES
('amit kumar', 10, '2021-01-10', 'mathematics', 'delhi', 40000),
('sunita devi', 20, '2020-03-15', 'commerce', 'mumbai', 35000),
('rahul sharma', 10, '2022-05-22', 'mathematics', 'delhi', 42000),
('priya nair', 30, '2019-11-01', 'science', 'kochi', 50000),
('aniket das', 20, '2023-02-14', 'commerce', 'kolkata', 32000);
```

(b) Give Increment of 25% salary for Mathematics Department.

```
START TRANSACTION;
UPDATE Teacher
SET SalaryAmt = SalaryAmt * 1.25
WHERE DeptName = 'mathematics';
SELECT * FROM Teacher WHERE DeptName = 'mathematics';
```

TeacherName	DeptNo	DateOfJoining	DeptName	LocationName	SalaryAmt
amit kumar	10	2021-01-10	mathematics	delhi	50000
rahul sharma	10	2022-05-22	mathematics	delhi	52500

(c) Perform Rollback command.

```
ROLLBACK;
SELECT * FROM Teacher WHERE DeptName = 'mathematics';
```

TeacherName	DeptNo	DateOfJoining	DeptName	LocationName	SalaryAmt
amit kumar	10	2021-01-10	mathematics	delhi	40000
rahul sharma	10	2022-05-22	mathematics	delhi	42000

(d) Give increment of 15% salary for Commerce Department.

```
START TRANSACTION;
UPDATE Teacher
SET SalaryAmt = SalaryAmt * 1.15
WHERE DeptName = 'commerce';
```

(e) Perform Commit command.

```
COMMIT;
SELECT * FROM Teacher;
SET SQL_SAFE_UPDATES = 1;
```

	TeacherName	DeptNo	DateOfJoining	DeptName	LocationName	SalaryAmt
▶	amit kumar	10	2021-01-10	mathematics	delhi	40000
	sunita devi	20	2020-03-15	commerce	mumbai	40250
	rahul sharma	10	2022-05-22	mathematics	delhi	42000
	priya nair	30	2019-11-01	science	kochi	50000
	aniket das	20	2023-02-14	commerce	kolkata	36800

Lab Experiment 8

AIM: Create Sales table with the following fields (Sales No, Salesname, Branch, Salesamount, DOB)

a) Insert five records.
b) Calculate total salesamount in each branch.
c) Calculate average salesamount in each branch.
d) Display all the salesmen, DOB who are born in the month of December as day in character format i.e. 21-Dec-09.
e) Display the name and DOB of salesman in alphabetical order of the month.

SOLUTION CODE: (By MAHIN-2401010123)

```
CREATE DATABASE IF NOT EXISTS sales_db;
USE sales_db;

DROP TABLE IF EXISTS Sales;
CREATE TABLE Sales (
   SalesNo INT PRIMARY KEY,
   SalesName VARCHAR(50),
   BranchName VARCHAR(50),
   SalesAmount INT,
   DateOfBirth DATE
);
```

(a) Insert five records.

```
INSERT INTO Sales (SalesNo, SalesName, BranchName, SalesAmount, DateOfBirth) VALUES
(101, 'arnav gupta', 'north', 45000, '1995-12-21'),
(102, 'sana sheikh', 'south', 32000, '1998-05-15'),
(103, 'kabir malhotra', 'north', 55000, '1996-12-09'),
(104, 'isha patel', 'east', 40000, '1997-12-05'),
(105, 'vihaan nair', 'south', 28000, '1999-08-22');
```

(b) Calculate total salesamount in each branch.

```
SELECT BranchName, SUM(SalesAmount) AS TotalSales
FROM Sales
GROUP BY BranchName;
```

BranchName	TotalSales
north	100000
south	60000
east	40000

(c) Calculate average salesamount in each branch.

```
SELECT BranchName, AVG(SalesAmount) AS AverageSales
FROM Sales
GROUP BY BranchName;
```

	BranchName	AverageSales
▶	north	50000.0000
	south	30000.0000
	east	40000.0000

(d) Display all the salesmen, DOB who are born in the month of December as day in character format i.e. 21-Dec-09.

```
SELECT SalesName, DATE_FORMAT(DateOfBirth, '%d-%b-%y')
AS FormattedDOB
FROM Sales
WHERE MONTH(DateOfBirth) = 12;
```

	SalesName	FormattedDOB
▶	arnav gupta	21-Dec-95
	kabir malhotra	09-Dec-96
	isha patel	05-Dec-97

(e) Display the name and DOB of salesman in alphabetical order of the month.

```
SELECT SalesName, DateOfBirth
FROM Sales
ORDER BY MONTHNAME(DateOfBirth) ASC;
```

	SalesName	DateOfBirth
▶	vihaan nair	1999-08-22
	arnav gupta	1995-12-21
	kabir malhotra	1996-12-09
	isha patel	1997-12-05
	sana sheikh	1998-05-15

Lab Experiment 9

AIM: Consider the following tables namely “DEPARTMENTS” and “EMPLOYEES” Their schemas are as follows:

Departments (dept no , dept name , dept_location);
Employees (emp_id , emp_name , emp_salary,dept_no);

a) Develop a query to grant all privileges of employees table into departments table.
b) Develop a query to grant some privileges of employees table into departments table.
c) Develop a query to revoke all privileges of employees table from departments table.
d) Develop query to revoke some privileges of employees table from departments table.
e) Write a query to implement the save point.

SOLUTION CODE: (By MAHIN-2401010123)

```
CREATE DATABASE IF NOT EXISTS company_records;
USE company_records;

CREATE TABLE IF NOT EXISTS Departments (
   DeptNo INT PRIMARY KEY,
   DeptName VARCHAR(50),
   DeptLocation VARCHAR(50)
);
CREATE TABLE IF NOT EXISTS Employees (
   EmpId INT PRIMARY KEY,
   EmpName VARCHAR(50),
   EmpSalary INT,
   DeptNo INT,
   FOREIGN KEY (DeptNo) REFERENCES Departments(DeptNo)
);
CREATE USER IF NOT EXISTS 'dept_manager'@'localhost' IDENTIFIED BY 'secure_pass123';
```

(a) Develop a query to grant all privileges of employees table into departments table.

```
GRANT ALL PRIVILEGES ON company_records.Employees TO 'dept_manager'@'localhost';
```

(b) Develop a query to grant some privileges of employees table into departments table.

```
GRANT SELECT, UPDATE ON company_records.Employees TO 'dept_manager'@'localhost';
```

(c) Develop a query to revoke all privileges of employees table from departments table.

```
REVOKE ALL PRIVILEGES ON company_records.Employees FROM 'dept_manager'@'localhost';
```

(d) Develop a query to revoke some privileges of employees table from departments table.

```
REVOKE UPDATE ON company_records.Employees FROM 'dept_manager'@'localhost';
```

(e) Write a query to implement the save point.

```
START TRANSACTION;

INSERT INTO Employees (EmpId, EmpName, EmpSalary, DeptNo)
VALUES (501, 'rajat verma', 45000, NULL);
SAVEPOINT entry_alpha;
UPDATE Employees SET EmpSalary = 48000 WHERE EmpId = 501;
COMMIT;
```

	user	host
▶	dept_manager	localhost
	mysql.infoschema	localhost
	mysql.session	localhost
	mysql.sys	localhost
	root	localhost
	user1	localhost

	Grants for dept_manager@localhost
▶	GRANT USAGE ON *.* TO `dept_manager`@`l...
	GRANT SELECT ON `company_records`.`emplo...

Lab Experiment 10

AIM: Using the tables "DEPARTMENTS" and "EMPLOYEES" from Lab 9 perform the following queries:

a) Display the employee details, departments that the departments are same in both the emp and dept.
b) Display the employee name and Department name by implementing a left outer join.
c) Display the employee name and Department name by implementing a right outer join.
d) Display the details of those who draw the salary greater than the average salary.

SOLUTION CODE: (By MAHIN-2401010123)

```
CREATE DATABASE IF NOT EXISTS company_joins;
USE company_joins;

CREATE TABLE IF NOT EXISTS Departments (
   DeptNo INT PRIMARY KEY,
   DeptName VARCHAR(50),
   DeptLocation VARCHAR(50)
);
CREATE TABLE IF NOT EXISTS Employees (
   EmpId INT PRIMARY KEY,
   EmpName VARCHAR(50),
   EmpSalary INT,
   DeptNo INT
);
INSERT INTO Departments (DeptNo, DeptName, DeptLocation) VALUES
(10, 'mathematics', 'block a'),
(20, 'commerce', 'block b'),
(30, 'science', 'block c'),
(40, 'humanities', 'block d');

INSERT INTO Employees (EmpId, EmpName, EmpSalary, DeptNo) VALUES
(701, 'amit kumar', 45000, 10),
(702, 'sunita devi', 55000, 20),
(703, 'rahul sharma', 38000, 10),
(704, 'priya nair', 60000, 30),
(705, 'vicky singh', 42000, NULL);
```

(a) Display the employee details, departments that the departments are same in both the emp and dept.

```
SELECT E.*, D.DeptName, D.DeptLocation
FROM Employees E
JOIN Departments D ON E.DeptNo = D.DeptNo;
```

EmpId	EmpName	EmpSalary	DeptNo	DeptName	DeptLocation
701	amit kumar	45000	10	mathematics	block a
702	sunita devi	55000	20	commerce	block b
703	rahul sharma	38000	10	mathematics	block a
704	priya nair	60000	30	science	block c

(b) Display the employee name and Department name by implementing a left outer join.

```
SELECT E.EmpName, D.DeptName
FROM Employees E
LEFT OUTER JOIN Departments D ON E.DeptNo = D.DeptNo;
```

EmpName	DeptName
amit kumar	mathematics
sunita devi	commerce
rahul sharma	mathematics
priya nair	science
vicky singh	NULL

(c) Display the employee name and Department name by implementing a right outer join.

```
SELECT E.EmpName, D.DeptName
FROM Employees E
RIGHT OUTER JOIN Departments D ON E.DeptNo = D.DeptNo;
```

EmpName	DeptName
rahul sharma	mathematics
amit kumar	mathematics
sunita devi	commerce
priya nair	science
NULL	humanities

(d) Display the details of those who draw the salary greater than the average salary.

```
SELECT * FROM Employees
WHERE EmpSalary > (SELECT AVG(EmpSalary) FROM Employees);
```

EmpId	EmpName	EmpSalary	DeptNo
702	sunita devi	55000	20
704	priya nair	60000	30
NULL	NULL	NULL	NULL